

Deep Research: Avatrada 57 Topic 021

Engineering Report: Order Timeout Logic

Component: Time-In-Force Manager (Order Timeout Logic) **Reference:** avatrada_57.pdf , Section 21 **Analysis Date:** October 26, 2023

1. Technical Deconstruction

The system component is a "Time-In-Force" (TIF) manager responsible for enforcing a maximum lifetime on open orders. This is a client-side risk management mechanism, distinct from exchange-native TIF settings like `Good-Til-Canceled` (GTC) or `Immediate-Or-Cancel` (IOC).

The core specifications are: * **Limit Orders:** Timeout and cancellation after 120 seconds. * **Market Orders:** Timeout and cancellation after 30 seconds.

This functionality is critical for preventing "stale" orders. A stale order might exist because of a disconnected session, a frozen market, or an intended short-term strategy that is no longer valid. Automatically canceling these orders limits unintended exposure.

Core Architectural Components

1. **Order State Machine:** An order must transition through a well-defined set of states. A minimal state machine would be: `PENDING_SUBMIT` -> `ACTIVE` -> `FILLED` / `PARTIALLY_FILLED` / `CANCELLED`. To handle the timeout race condition, we must introduce an intermediate state: `PENDING_CANCEL`.
2. **Order Registry:** A centralized, thread-safe data structure (e.g., a dictionary or hash map) that stores all `ACTIVE` orders, indexed by their unique order ID. This registry is the single source of truth for the bot's view of open orders.

3. **Scheduler/Timer Mechanism:** A background process responsible for triggering the cancellation logic at the precise timeout deadline for each order. This must be efficient and capable of managing potentially thousands of concurrent order timers.
4. **Exchange Interface:** A module responsible for communicating with the exchange's API, specifically for sending `cancel` requests and receiving execution reports (fills, cancellations, rejections).
5. **Synchronization Primitive:** A locking mechanism (e.g., a mutex or semaphore) to ensure atomic updates to an order's state, preventing simultaneous writes from the fill-event handler and the timeout-event handler.

The Race Condition

The central challenge identified in the prompt is a classic race condition. It occurs when two asynchronous events—the timeout trigger and the fill notification from the exchange—attempt to modify the state of the same order concurrently.

Sequence of Events:

1. `T=0s` : A Limit order `LMT-123` is submitted. The TIF manager starts a 120s timer. The order's state is `ACTIVE` .
2. `T=120.000s` : The TIF manager's timer for `LMT-123` fires. The manager decides to cancel the order.
3. `T=120.001s` : The exchange matches the order. A fill notification is generated and sent to the bot over the network.
4. `T=120.005s` : The TIF manager sends a `CANCEL` request for `LMT-123` to the exchange.
5. `T=120.015s` : The fill notification for `LMT-123` arrives at the bot. The fill handler attempts to update the order's state to `FILLED` .

The Conflict: Without proper synchronization, the timeout logic and the fill handler will race to update the order's state. This can lead to an inconsistent state where the bot believes an order was canceled when it was actually filled, causing incorrect position and P&L calculations.

2. Implementation Strategy

We will design a robust solution in Python using the `asyncio` library, which is ideal for handling I/O-bound operations like network communication with a trading exchange.

Data Structures and Classes

First, we define the `Order` object, which includes state, an `asyncio.Lock` for synchronization, and other relevant details.

```
import asyncio
from enum import Enum, auto
from dataclasses import dataclass, field
import time

class OrderType(Enum):
    LIMIT = auto()
    MARKET = auto()

class OrderStatus(Enum):
    PENDING_SUBMIT = auto()
    ACTIVE = auto()
    PENDING_CANCEL = auto() # Crucial state for handling the race condition
    FILLED = auto()
    CANCELLED = auto()
    REJECTED = auto()

@dataclass
class Order:
    order_id: str
    order_type: OrderType
    status: OrderStatus = OrderStatus.ACTIVE
    submission_time: float = field(default_factory=time.time)
    lock: asyncio.Lock = field(default_factory=asyncio.Lock)

    def get_timeout(self) -> int:
        return 120 if self.order_type == OrderType.LIMIT else 30
```

The Time-In-Force Manager

The manager will run as a dedicated `asyncio` task. It will maintain a registry of active orders and use `asyncio.create_task` to spawn a timer for each new order.

```
class TimeInForceManager:
    def __init__(self, exchange_interface):
        self.active_orders = {} # {order_id: Order}
        self.exchange = exchange_interface
        print("TIF Manager Initialized.")

    async def add_order(self, order: Order):
        """Register a new order and start its timeout timer."""
        if order.order_id in self.active_orders:
            print(f"Warning: Order {order.order_id} already being tracked.")
            return

        print(f"Tracking order {order.order_id} with a {order.get_timeout()}s
timeout.")
        self.active_orders[order.order_id] = order
        asyncio.create_task(self._start_timer(order))

    async def _start_timer(self, order: Order):
        """The background timer task for a single order."""
        await asyncio.sleep(order.get_timeout())

        # --- RACE CONDITION CRITICAL SECTION START ---
        async with order.lock:
            # If order is still ACTIVE after the timeout, attempt to cancel.
            # If it was already filled or cancelled by another process, do
            nothing.

            if order.status == OrderStatus.ACTIVE:
                print(f"Timeout for order {order.order_id}. Issuing cancel
request.")

                order.status = OrderStatus.PENDING_CANCEL
                await self.exchange.send_cancel_request(order.order_id)
            else:
                print(f"Timeout for {order.order_id} fired, but status is
already {order.status}. No action taken.")
```

```

# --- RACE CONDITION CRITICAL SECTION END ---

async def on_fill_received(self, order_id: str):
    """Callback for when a fill notification arrives from the exchange."""
    if order_id not in self.active_orders:
        return

    order = self.active_orders[order_id]

    # --- RACE CONDITION CRITICAL SECTION START ---
    async with order.lock:
        # The fill "wins" the race if the state was ACTIVE or
        PENDING_CANCEL.
        if order.status in [OrderStatus.ACTIVE, OrderStatus.PENDING_CANCEL]:
            print(f"Fill received for {order.order_id}. State changed to
            FILLED.")
            order.status = OrderStatus.FILLED
            # The order is now in a terminal state, remove it from active
            tracking.
            del self.active_orders[order_id]
        else:
            # This case is unlikely but handles weird edge cases.
            print(f"Fill received for {order.order_id}, but status was
            {order.status}. Ignoring.")
    # --- RACE CONDITION CRITICAL SECTION END ---

    async def on_cancel_confirmed(self, order_id: str):
        """Callback for when the exchange confirms a cancellation."""
        if order_id not in self.active_orders:
            return

        order = self.active_orders[order_id]
        async with order.lock:
            if order.status == OrderStatus.PENDING_CANCEL:
                print(f"Cancel confirmation for {order.order_id}. State changed
                to CANCELLED.")
                order.status = OrderStatus.CANCELLED
                del self.active_orders[order_id]

    async def on_cancel_rejected(self, order_id: str, reason: str):

```

```

        """Callback for when the exchange rejects a cancellation."""
        # This is expected if the fill won the race condition.
        if "already filled" in reason.lower():
            print(f"Cancel for {order_id} rejected because it was already
filled. This is an expected race condition outcome.")
            # The on_fill_received handler will manage the state change.
        else:
            print(f"CRITICAL: Cancel for {order_id} rejected for unexpected
reason: {reason}")
            # Here, you would add alerting or error handling logic.

```

Solution to the Race Condition

The solution hinges on two elements: 1. **The `PENDING_CANCEL` State:** This intermediate state signifies our *intent* to cancel. It allows the system to differentiate between an order that is live and an order that is in the process of being canceled due to a timeout. 2. **The `asyncio.Lock`:** By wrapping all state modifications for a given order inside an `async with order.lock:` block, we guarantee that only one coroutine (either the timer or the fill handler) can modify the order's status at any given moment. This enforces atomicity and resolves the race condition deterministically.

How it works:

- * If Timeout Wins:** The `_start_timer` coroutine acquires the lock first. It sees the state is `ACTIVE`, changes it to `PENDING_CANCEL`, and sends the cancel request. When the `on_fill_received` coroutine eventually runs, it will acquire the lock, see the state is `PENDING_CANCEL`, and correctly transition it to `FILLED`. The subsequent "Cancel Rejected" message is expected and can be safely logged.
- * If Fill Wins:** The `on_fill_received` coroutine acquires the lock first. It sees the state is `ACTIVE`, changes it to `FILLED`, and removes the order from tracking. When the `_start_timer` coroutine eventually runs, it will acquire the lock, see the state is now `FILLED` (not `ACTIVE`), and will do nothing, correctly aborting the cancellation attempt.

3. Critical Analysis

Potential Failure Modes & Edge Cases

- 1. System Crash and Restart (Persistence):** The current in-memory implementation will lose all state upon a crash. If the bot restarts, it will not be tracking previously active orders, which may remain live on the exchange indefinitely.
 - **Mitigation:** Implement persistence. Before submitting an order, write its details (ID, submission time) to a durable store (e.g., SQLite, Redis). On startup, the TIF manager must load all non-terminal orders from the store and resume their timeout timers. The timeout duration must be recalculated as `original_timeout - (time.now() - submission_time)`.
- 2. Clock Skew:** The bot's system clock may drift from the exchange's clock. This could cause cancellations to be sent slightly earlier or later than intended from the exchange's perspective.
 - **Mitigation:** For a client-side TIF, this is generally acceptable as the timeout is a rule enforced by the client. However, for high-precision requirements, the system should synchronize its clock with an NTP server. Rely on exchange-provided timestamps in execution reports for auditing and logging.
- 3. Network Latency:** Significant latency between the bot and the exchange can widen the window for the race condition. A cancel request could be in-flight for several hundred milliseconds, during which a fill can occur.
 - **Mitigation:** The proposed state machine (`PENDING_CANCEL`) is designed specifically to handle this. The key is to trust the exchange's report: if the exchange confirms a fill, it was filled, regardless of our intent to cancel.

4. **Partial Fills:** The logic assumes a binary `FILLED` or `NOT FILLED` state. Orders can be partially filled.

- **Mitigation:** The specification should be clarified. The most common interpretation is to cancel the *remaining, unfilled quantity* of the order upon timeout. The `Order` object should track `quantity` and `filled_quantity`. The `on_fill_received` handler would update `filled_quantity`. The timeout logic would then issue a `Cancel` request, which on most exchanges automatically targets the remaining quantity.

5. **Aggressive Market Order Timeout (30s):** A market order is designed to fill almost instantaneously. A 30-second timeout implies one of two scenarios:

- **Handling Illiquidity:** In extremely thin markets, a market order might not find a counter-party immediately.
- **Handling Exchange Failure:** The timeout acts as a safety net if the exchange is unresponsive or has lost the order. This is a valid, albeit aggressive, risk control. The implementation should be prepared to handle the cancellation of market orders, which some exchanges may not permit.

Optimizations

- **Scheduler Efficiency:** The current `asyncio.create_task` approach spawns one task per order. For tens of thousands of orders, this is still highly efficient. An alternative for extreme scale would be to manage a single sorted list or min-heap of `(expiry_time, order_id)` tuples and have a single master coroutine that sleeps until the time of the next expiry. `asyncio`'s event loop does something similar under the hood, so the current approach is robust and Pythonic.
- **API Rate Limits:** If canceling many orders simultaneously, the system could hit the exchange's API rate limits.
 - **Mitigation:** The `exchange_interface` should implement a token bucket or similar rate-limiting algorithm to throttle outgoing cancel requests, queuing them if necessary.